IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

ServiceNow Micro Project Report

1. INTRODUCTION

1.1 Project Overview

This project demonstrates the use of ServiceNow UI Policies and Client Scripts to control Incident form behavior. It improves data accuracy by making fields mandatory, read-only, automatically updating values, and validating data before saving.

1.2 Purpose

The purpose is to ensure that Incident records contain complete, correct, and consistent information and to reduce manual errors during incident management.

2. IDEATION PHASE

2.1 Problem Statement

Manual data entry in Incident records can result in incomplete or incorrect information. This may affect triage, routing, reporting, SLA compliance, and service quality.

2.2 Empathy Map Canvas

Says: “I need to create an incident quickly.”
Thinks: “Are all required fields filled?”
Does: Creates and updates incidents; enters Incident details.
Feels: Wants a simple and reliable process and fewer data-entry errors.

2.3 Brainstorming

Possible solutions include using UI Policies for dynamic field control, Client Scripts for automatic updates, validation before submission, prevention of unauthorized list editing, and automatic Urgency setting for High Impact incidents.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey map

Create Incident → Enter Details → Select Impact → System Applies Rules → Validate Fields → Submit Incident → Incident Saved. If required information is missing, an error is displayed and submission is prevented.

3.2 Solution Requirement

The system should make required fields mandatory when needed, make Urgency read-only for High Impact, automatically set Urgency to High, prevent saving when Assigned To is missing, block State changes through list editing, and allow State changes through the Incident form.

3.3 Data Flow Diagram

User → Incident Form → Enter/Change Data → UI Policy + Client Scripts → Validation & Field Control → Valid Incident Record → Save.
If the data is invalid, the system shows an error and prevents saving.

3.4 Technology Stack

ServiceNow; Incident Management; UI Policy; UI Policy Actions; Client Scripts; JavaScript; Form Validation.

4. PROJECT DESIGN

4.1 Problem Solution Fit

Missing information: Mandatory field validation.
Incorrect urgency: Automatic urgency update.
Unwanted field modification: Read-only control.
Invalid submission: onSubmit validation.
Unauthorized list editing: onCellEdit Client Script.

4.2 Proposed Solution

The proposed solution combines UI Policies and Client Scripts to dynamically control Incident records based on the Impact value. For High Impact incidents, additional controls are automatically applied.

4.3 Solution Architecture

User → ServiceNow Incident → UI Policy + Client Scripts → Field Control / Validation / Automation → Valid Incident Record → Save.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

1. Identify the Incident data-entry problem.
2. Create the High Impact UI Policy.
3. Configure the Urgency UI Policy Action.
4. Create the onChange Client Script.
5. Create the onSubmit Client Script.
6. Create the onCellEdit Client Script.
7. Test all configurations.
8. Document the results.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Test 1: Impact = High → Assigned To becomes required.
Test 2: Impact = High → Urgency becomes read-only.
Test 3: Impact = High → Urgency is automatically set to High.
Test 4: High Impact + no Assigned To → Save is prevented.
Test 5: State edited from list → Change is blocked.
Test 6: State edited from form → Change is allowed.
Test 7: Impact changed to Medium → Restrictions are reversed.

7. RESULTS

7.1 Output Screenshots

The ServiceNow Incident list displays Incident records with Number, Opened, Short Description, Caller, Priority, State, Category, Assignment Group, Assigned To and Updated fields. The implementation demonstrates High Impact controls, automatic Urgency update, mandatory validation, read-only behavior, save-time validation and State editing restriction.

8. ADVANTAGES & DISADVANTAGES

Advantages

• Improves data accuracy.
• Reduces manual errors.
• Provides automatic field control.
• Prevents invalid submissions.
• Improves consistency of Incident records.
• Lightweight and easy to implement.

Disadvantages

• Requires ServiceNow configuration knowledge.
• Client-side scripts need maintenance when requirements change.
• Incorrect configuration may affect user interaction.

9. CONCLUSION

The project demonstrates how ServiceNow UI Policies and Client Scripts can control Incident form behavior, automate updates, validate information, and prevent incorrect data submission. The combination of onChange, onSubmit, and onCellEdit scripts improves data integrity and form usability.

10. FUTURE SCOPE

• Add advanced Incident validation rules.
• Integrate automated notifications.
• Add dashboards for Incident monitoring.
• Implement more workflow automation.
• Integrate AI-based Incident classification and prioritization.
• Improve reporting and analytics.

11. APPENDIX

Source Code (if any)

The project contains JavaScript Client Scripts for automatic Urgency update, Assigned To validation, and State list-edit restriction.

Dataset Link

Not Applicable — this project does not use a separate dataset.

GitHub & Project Demo Link

GitHub: https://github.com/Reeshab16
ServiceNow Project Demo: https://nowlearning-nlinst04738622-8aq8u-0001.lab.service-now.com/now/nav/ui/classic/params/target/incident_list.do%3Fsysparm_clear_stack%3Dtrue
Note: The ServiceNow demo link requires login/access to the instance.